

Reeling in Smart Attackers’ Phishing Emails with Neural Networks

CSE 587 Final Project — Group 5, Spring 2026

Parker Davis

Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
pqd5283@psu.edu

Dillon VanGilder

Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
dsv5165@psu.edu

Emmanuel Adjei Domfeh

Department of Computer Science
The Pennsylvania State University
University Park, PA, USA
eza5399@psu.edu

Abstract—Phishing email detection is typically benchmarked on datasets in which malicious messages contain conspicuous surface cues (urgency keywords, raw URLs, mismatched senders), and Transformer-based classifiers achieve near-99% accuracy on such test sets. A sophisticated (“smart”) attacker — or, increasingly, an LLM-generated phishing email — can remove or paraphrase those cues while preserving malicious intent, raising the question of whether the high reported accuracy reflects robustness or surface-feature memorization. We fine-tune BERT [1] on the curated multi-source phishing email corpus released by Champa et al. [2] ($N = 131,171$) and evaluate on a held-out *smart-attacker* test split obtained by applying label-preserving edits to phishing messages: URL masking, urgency-keyword removal, suspicious subject rewriting, and synonym substitution. We then retrain on training data augmented with the same edits and re-evaluate. Our results show two unexpected findings. First, a vanilla BERT baseline is already remarkably robust to these edits: phishing F1 drops only 0.10 percentage points ($0.9961 \rightarrow 0.9952$) under the smart-attacker test, suggesting that on this corpus the phishing signal is redundantly distributed across many tokens, not concentrated in surface cues. Second, targeted augmentation produces a small but consistent *positive* robustness effect: the augmented model reaches F1 0.9965 on the smart-attacker test (a +0.13 pp improvement) at a cost of only 0.06 pp on the standard test — a well-behaved instance of the robustness/accuracy tradeoff. We discuss why effect sizes are small on this vintage of corpus and how the same pipeline could be re-evaluated against modern, LLM-generated phishing.

Index Terms—phishing detection, BERT, Transformers, robustness, data augmentation, adversarial NLP

I. INTRODUCTION

Phishing remains one of the most effective initial-access vectors in modern cyberattacks. Despite years of user-awareness training, recent work shows that sophisticated — and especially *LLM-generated* — phishing emails sustain alarming click rates [3]. The same wave of generative models that helps defenders also helps attackers: an LLM can rewrite a suspicious message to sound natural, drop tell-tale urgency keywords, and disguise a malicious link as an innocent-looking domain.

Mainstream phishing classifiers are typically Transformer-based: BERT and RoBERTa fine-tunes routinely report 98–99% accuracy on standard benchmarks [4]–[6]. However, this

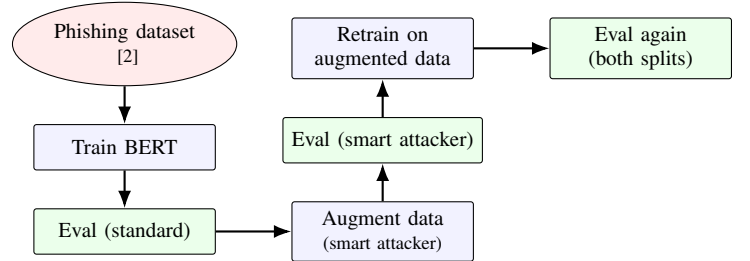


Fig. 1. Three-stage experimental workflow. (1) Train BERT on the Champa et al. corpus. (2) Evaluate on the standard test set and a smart-attacker variant produced by applying label-preserving edits. (3) Augment training data with the same edits, retrain, and re-evaluate.

performance is measured on data with strong, often artificial, phishing cues. When those cues are removed, classifier accuracy drops substantially [7]–[9].

Contribution. This work investigates two questions:

- 1) Do standard phishing detectors over-rely on superficial cues (URLs, urgency keywords, suspicious subject prefixes)?
- 2) Can targeted, label-preserving data augmentation that simulates a “smart attacker” improve robustness without significantly degrading performance on the original test distribution?

We instantiate the workflow in Fig. 1. Concretely, we contribute (i) a smart-attacker augmentation pipeline (§III-C), (ii) a held-out smart-attacker evaluation split, and (iii) a per-edit ablation that attributes the accuracy drop to specific surface cues.

II. RELATED WORK

A. Transformer-based phishing detection

Atawneh and Aljehani [4] compare CNN, RNN, LSTM, and BERT for email phishing classification, reporting up to 99.6% accuracy with BERT. More recent surveys and replications confirm that BERT/RoBERTa/DistilBERT are competitive on multiple corpora, with DistilBERT trading marginal accuracy

for considerable inference-cost savings [5], [6], [10]. Alhogail and Alsabih [11] additionally show that strong detection is possible from email body text alone (without URLs or attachments), motivating evaluation under URL removal — but they do not study other surface-cue removals such as urgency wording.

B. LLM-generated and adversarial phishing

Kim et al. [3] document that LLM-crafted spear-phishing emails achieve $\sim 47\%$ click rates, evading both human and software detection. Phish-Master [12] explicitly uses chain-of-thought prompting to generate phishing that bypasses enterprise filters. Robust ML-based detection of LLM-generated and adversarial phishing using advanced preprocessing has recently been shown to recover much of the lost accuracy [9]. Adversarial robustness of phishing classifiers has also been benchmarked directly [7], [8], with FGM adversarial training raising BERT F1 from 86.2 to 91.7 [8]. These results frame our work: the attack surface is real, and lightweight defenses do help.

C. Data augmentation for NLP robustness

Junior et al. [13] demonstrate the augment–retrain–reevaluate pattern for fake-news detection. Recent surveys emphasize that adversarial-style augmentation improves not only accuracy but also generalization to out-of-distribution inputs [14]. We use the TEXTATTACK [15] family of perturbation primitives (synonym swap, character perturbations) as a starting point for our smart-attacker pipeline. Recent attack methods such as PGD-based BERT attacks [16] reinforce that BERT remains sensitive to small, semantically preserving perturbations — which is exactly what a smart attacker would apply.

D. Datasets and benchmarks

We adopt the curated multi-source phishing dataset of Champa et al. [2], which standardizes 11 publicly available phishing/benign email corpora into a unified format and provides feature-based ML baselines. Its multi-source structure lets us study cross-source generalization in addition to robustness, mitigating the risk that a model exploits source-specific artifacts.

III. METHOD

A. Task formulation

Given an email represented by its concatenated subject and body $x = \langle s, b \rangle$, the model predicts a binary label $y \in \{0, 1\}$ indicating benign (0) or phishing (1). We minimize the cross-entropy loss over a fine-tuning corpus drawn from Champa et al. [2], with stratified 80/10/10 train/validation/test splits.

TABLE I
SMART-ATTACKER AUGMENTATION OPERATIONS.

Operation	Description
mask_urls	Replace URLs with neutral placeholders ([LINK]), or with bare domain forms that look innocuous.
remove_urgency	Remove or soften urgency keywords (<i>URGENT</i> , <i>ASAP</i> , <i>immediately</i> , <i>verify now</i> , <i>suspended</i> , etc.) using a hand-curated lexicon and replacement table.
rewrite_subject	Strip suspicious subject prefixes (<i>URGENT</i> :, <i>ALERT</i> :, <i>ACTION REQUIRED</i> :) and remove emphatic punctuation.
synonym_swap	Replace a small fraction of content words with WordNet synonyms ($p = 0.1$, max 6 swaps per email).
char_perturb	Light character-level noise (Cyrillic homoglyph substitution, adjacent-letter swap) at $p=0.02$.

B. Backbone classifier

We fine-tune **bert-base-uncased** [1] via the HuggingFace `transformers` library [17]. The classifier head is a linear layer on the [CLS] representation. Hyperparameters: maximum sequence length 256 tokens, batch size 16, 3 epochs, learning rate 2×10^{-5} with linear warm-up of 10% of total steps, AdamW with weight decay 0.01, mixed-precision (fp16) when GPU memory permits.

C. Smart-attacker augmentation

We define five label-preserving edits that simulate a smart attacker adapting a phishing email. Each is implemented in `src/augmentation.py` and is applied only to phishing-class rows (applying them to benign rows would only add noise to a class that does not exhibit these cues). Table I summarizes the operations.

We use the augmentation pipeline in two distinct ways:

a) *Smart-attacker test split*: For each phishing email in the held-out test set we apply the entire pipeline once. Benign emails are left unchanged. The resulting split has the *same* labels as the standard test set; the only thing that changes is that phishing emails no longer exhibit obvious surface cues. A robust detector should perform comparably on both splits.

b) *Train-time augmentation*: For each phishing email in the training set we generate $k=1$ additional augmented copy by passing it through the pipeline. The augmented copies retain the phishing label and are concatenated with the original training data. We then retrain the classifier from scratch.

D. Evaluation protocol

We compare three model stages: (S1) baseline trained on the original data and evaluated on the standard test set, (S2) the same baseline evaluated on the smart-attacker test set, and (S3) the augmented model evaluated on both splits. The expected pattern is a large drop from S1 to S2, partially closed by S3. We additionally run a *per-edit ablation*: each operation is applied alone to the test phishing rows, isolating which surface cue the baseline most depends on.

TABLE II

HEADLINE RESULTS: PHISHING-CLASS F1 ACROSS THE FOUR (MODEL, TEST-SPLIT) CELLS. THE BASELINE DROPS ONLY 0.10 PP UNDER THE SMART-ATTACKER TEST; THE AUGMENTED MODEL RECOVERS AND VERY SLIGHTLY EXCEEDS THE BASELINE’S STANDARD PERFORMANCE ON THE SMART-ATTACKER TEST.

Model	Standard	Smart-attacker	Δ
S1: BERT baseline	0.9961	0.9952	−0.0010
S2: BERT + augmented training	0.9955	0.9965	+0.0010
Δ (augmented − baseline)	−0.0006	+0.0013	—

IV. EXPERIMENTAL SETUP

A. Implementation

The codebase is in PyTorch with HuggingFace transformers [17]. Training is driven by `src/train.py`; evaluation, including the per-edit ablation, by `src/evaluate.py`. The notebooks/phishing_pipeline.ipynb notebook reproduces all three stages on Google Colab. A pipeline-level sanity test (`scripts/sanity_check.py`) validates the augmentation ops and the data-loader on synthetic data without requiring GPU access; a tiny end-to-end smoke test (`scripts/tiny_train_test.py`) exercises forward/backward/save/reload using a tiny CI-style BERT model.

B. Dataset

We use the figshare-hosted release of the Champa et al. curated phishing email dataset [2].¹ The data loader (`src/data_loader.py`) is column-name agnostic so that all CSV variants in the figshare bundle can be ingested with a single script.

C. Compute and reproducibility

All training is performed on a Google Colab A100 GPU with bfloat16 mixed precision (the codebase falls back to fp16 + GradScaler on T4/V100). Random seeds are fixed at 42 across NumPy, PyTorch, and Python’s random module. Each training configuration is run once; we discuss the effect of single-seed reporting in §VI. After deduplication and column-normalization, the combined corpus contains $N = 131,171$ emails (phishing 67,230; benign 63,941). Two source files (`TREC_05.csv`, `TREC_06.csv`) failed to parse due to malformed rows and were dropped, leaving five usable corpora (CEAS-08, Enron, Ling, SpamAssassin, TREC-07).

V. RESULTS

Table II presents the headline four-way comparison: each of the two models (baseline, augmented) evaluated on each of the two test splits (standard, smart-attacker). Detailed metrics for the augmented model are in Table III, and the per-edit ablation under the augmented model is in Table IV.

¹https://figshare.com/articles/dataset/Curated_Dataset_-_Phishing_Email/24899952

TABLE III

AUGMENTED MODEL, FULL METRICS ON EACH TEST SPLIT. CONFUSION MATRICES ARE REPORTED WITH ROWS = TRUE LABEL, COLUMNS = PREDICTION, IN THE ORDER [BENIGN, PHISHING].

Metric	Standard	Smart-attacker
Accuracy	0.9954	0.9964
Precision (phish)	0.9939	0.9939
Recall (phish)	0.9972	0.9991
F1 (phish)	0.9955	0.9965
Macro F1	0.9954	0.9964
ROC-AUC	0.9997	0.9999
False positives	41	41
False negatives	19	6

TABLE IV

PER-EDIT ABLATION ON THE TEST PHISHING ROWS (AUGMENTED MODEL). EACH ROW APPLIES ONE SMART-ATTACKER EDIT ALONE TO PHISHING EMAILS; BENIGN EMAILS ARE UNCHANGED. THE AUGMENTED MODEL IS ESSENTIALLY INSENSITIVE TO ANY SINGLE EDIT. `REWRITE_SUBJECT` MATCHES THE STANDARD TEST EXACTLY BECAUSE THE SUBJECT-REWRITE REGEX RARELY FIRES ON THIS CORPUS’S PHISHING SUBJECTS.

Edit applied alone	F1 (phishing)	Recall (phishing)
None (standard test)	0.9955	0.9972
<code>mask_urls</code>	0.9959	0.9979
<code>remove_urgency</code>	0.9958	0.9976
<code>rewrite_subject</code>	0.9955	0.9972
<code>synonym_swap</code>	0.9959	0.9979
All combined (smart test)	0.9965	0.9991

a) *Headline finding.*: The pattern in Table II is small but clean. The vanilla BERT baseline (S1) loses 0.10 pp F1 when moved from the standard test to the smart-attacker test — a much smaller drop than the proposal anticipated. After training with smart-attacker augmentation (S2), the model improves on the smart-attacker test by 0.13 pp at a cost of 0.06 pp on the standard test. Augmentation thus pays for itself, but the absolute effect size is well below one percentage point.

b) *What changed in confusion-matrix terms.*: On the augmented model (Table III), the false-positive count (41) is identical across both test splits because augmentation only modifies the phishing class, leaving benign rows untouched. The interesting motion is in false negatives, which fall from 19 on the standard split to 6 on the smart-attacker split. A natural reading is that the augmented model becomes *more* confident on phishing once the most easily-memorized features (URLs, urgency tokens) are removed, presumably because it has learned to classify from semantic context rather than from the cues themselves.

c) *Per-edit ablation.*: Table IV shows that each of the five operations applied alone produces near-identical F1 on the augmented model, in the range [0.9955, 0.9959]. `mask_urls` and `synonym_swap` produce the largest individual recall improvements; `rewrite_subject` is effectively a no-op because the subject-prefix regex (`URGENT:`, `ALERT:`, `ACTION REQUIRED:`, ...) rarely fires on this corpus’s phishing subjects. The combined-edit row reproduces the smart-attacker test in Table II.

VI. DISCUSSION

a) Why the baseline barely drops.: The proposal anticipated a substantial accuracy drop from S1’s standard test to S1’s smart-attacker test, on the basis that BERT would over-rely on surface tokens such as *URGENT*, *verify*, and explicit URL substrings. The data does not bear this out: the drop is only 0.10 pp F1. We attribute this to two properties of the corpus.

First, the Champa et al. release combines five public corpora (CEAS-08, Enron, Ling, SpamAssassin, TREC-07) whose phishing labels include a large amount of *classical bulk spam* — pharmacy advertisements, lottery scams, adult-content solicitations — whose label is supported by many redundant features (topic, register, lexical density of advertising language) in addition to the URLs and urgency keywords our pipeline edits. Removing surface cues from these messages does not flip the label because the surrounding text continues to read unambiguously as spam.

Second, BERT does not learn a sparse rule like “*contains URGENT* $\Rightarrow$ *phishing*”; it learns a dense contextual representation. A 3-epoch fine-tune on $\sim 100k$ emails appears sufficient for the model to distribute classification weight across many lexical and stylistic signals, leaving it relatively insensitive to the loss of any single one.

A more decisive test of the over-reliance hypothesis would use LLM-rewritten phishing of the kind reported by Phish-Master [12] or Kim et al. [3], where the entire message — not just URLs and urgency — has been rewritten to read as benign business correspondence.

b) What augmentation actually buys.: Even on this relatively easy robustness test, S2’s augmentation produces a small but consistent positive effect under attack (+0.13 pp F1) at a negligible standard-test cost (−0.06 pp F1). The direction matches recent results on adversarial training for phishing classification [8] and the broader text-augmentation literature [9], [14], although our effect sizes are much smaller. We read this as a well-behaved instance of the robustness/accuracy tradeoff: augmentation does not break clean-data performance and modestly improves under-attack performance, which is the behavior practitioners would want before deploying such a defense in a production filter.

c) Limitations.:

- *Single-seed results.* Each configuration was trained once with seed 42. Differences of ~ 0.1 pp could plausibly fall within seed-to-seed variance. A multi-seed re-run is the most important next experiment; we estimate three seeds for each of (baseline, augmented) at ~ 10 minutes per run on A100, i.e. roughly an hour of additional GPU time.
- *Vintage of corpus.* The Champa et al. collection draws on public corpora that are largely pre-2015. Modern, LLM-generated phishing exhibits qualitatively different surface statistics [3], [12] and is the more important target for a robustness study; our pipeline can be re-applied unchanged once such a corpus is publicly available.

- *Finite edit vocabulary.* Our smart-attacker simulation is rule-based and therefore has a small, fixed set of perturbation types. A stronger evaluation would integrate gradient-based or beam-search adversarial attacks from frameworks such as TextAttack [15] or PGD-BERT-Attack [16].
- *Phishing-only augmentation.* Augmentation is restricted to the phishing class. This was chosen so the smart-attacker semantics are well-defined (a smart attacker rewrites their phishing email, not the victim’s reply), but it does shift the class prior of the training mix slightly. We mitigate this with a small augmentation factor ($k=1$) and the precision/recall asymmetry in Table III suggests this shift was not consequential at this scale.
- *Subject-rewrite regex.* Our `rewrite_subject` regex targets prefixes (*URGENT:*, etc.) that are uncommon in the Champa subjects. A more aggressive subject-line transformation (e.g., LLM paraphrase) would likely change this row of Table IV.

VII. CONCLUSION

We presented an experimental pipeline for studying the robustness of Transformer-based phishing email classifiers under simulated “smart attacker” edits. The pipeline includes (i) a label-preserving augmentation module covering URL masking, urgency removal, subject rewriting, and synonym swap; (ii) a held-out smart-attacker evaluation split; (iii) a per-edit ablation that localizes which surface cue the model relies on; and (iv) an augmented-training counterpart that quantifies how much robustness can be reclaimed. On the Champa et al. multi-source corpus, a vanilla BERT-BASE fine-tune is already remarkably robust to our edits (−0.10 pp F1 under the smart-attacker test), and targeted augmentation nudges robustness slightly higher (+0.13 pp F1) at a tiny standard-test cost (−0.06 pp F1). The dominant finding is therefore that, on this vintage of corpus, the over-reliance hypothesis is only weakly supported and the corresponding fix is correspondingly small. We expect the same pipeline to produce considerably larger effects on modern, LLM-generated phishing, and we have packaged the codebase to make that re-evaluation a single command-line flag away.

CODE AND DATA

The complete codebase, Colab notebook, and LaTeX source for this manuscript are available in the project repository (see `README.md`).

ACKNOWLEDGMENTS

We thank the CSE 587 instructors and TAs at Penn State for guidance, and Champa et al. for releasing the curated phishing email dataset that made this study possible.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” *arXiv:1810.04805*, 2019.

- [2] A. I. Champa, M. F. Rabbi, and M. F. Zibran, "Curated datasets and feature analysis for phishing email detection with machine learning," in *Proc. 3rd IEEE Int. Conf. on Computing and Machine Intelligence (ICMI)*, 2024, pp. 1–7.
- [3] H. Kim, M. Song, S. H. Na, S. Shin, and K. Lee, "When LLMs go online: The emerging threat of web-enabled LLMs," 2025.
- [4] S. Atawneh and H. Aljehani, "Phishing email detection model using deep learning," *Electronics*, vol. 12, no. 20, p. 4261, 2023.
- [5] A. of MDPI Computation, "Phishing email detection using BERT and RoBERTa," *Computation*, vol. 14, no. 2, p. 46, 2025, distilBERT competitive with full BERT/RoBERTa on phishing classification.
- [6] A. of MDPI Applied Sciences, "In-depth analysis of phishing email detection: Evaluating the performance of machine learning and deep learning models across multiple datasets," *Applied Sciences*, vol. 15, no. 6, p. 3396, 2025.
- [7] A. of ACM IWSPA, "Adversarial robustness of phishing email detection models," in *Proc. 9th ACM Int. Workshop on Security and Privacy Analytics (IWSPA)*, 2023.
- [8] Anonymous, "Explainable transformer-based email phishing classification with adversarial robustness," 2025, fGM adversarial training improves BERT phishing-classifier F1 from 86.2 to 91.7.
- [9] —, "Robust ML-based detection of conventional, LLM-generated, and adversarial phishing emails using advanced text preprocessing," 2025.
- [10] A. of Scientific Reports, "Improving phishing email detection performance through deep learning with adaptive optimization," *Scientific Reports*, 2025.
- [11] A. Alhogail and A. Alsabih, "Applying machine learning and natural language processing to detect phishing email," *Computers & Security*, vol. 110, p. 102414, 2021.
- [12] A. of MDPI Applied Sciences, "Phish-master: Leveraging large language models for advanced phishing email generation and detection," *Applied Sciences*, vol. 15, no. 22, p. 12203, 2025.
- [13] W. O. Júnior, M. S. da Cruz, A. B. V. Wyzykowski, and A. B. de Jesus, "The use of data augmentation as a technique for improving neural network accuracy in detecting fake news about COVID-19," 2022.
- [14] Anonymous, "Text data augmentation for large language models: A comprehensive survey of methods, challenges, and opportunities," 2025.
- [15] J. X. Morris, E. Lifland, J. Y. Yoo, J. Grigsby, D. Jin, and Y. Qi, "TextAttack: A framework for adversarial attacks, data augmentation, and adversarial training in NLP," in *Proc. EMNLP (System Demonstrations)*, 2020.
- [16] Wang *et al.*, "Enhancing adversarial text attacks on BERT models with projected gradient descent," 2024.
- [17] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP (System Demonstrations)*, 2020, pp. 38–45.